

MEDSCOPE-AI

Medical Document Intelligence & Patient Timeline Platform

Turn fragmented medical records into one understandable, evidence-backed patient journey: upload, classify, extract (precision-faithful dates), timeline + episodes + graph, evidence + conflicts + gaps + What Changed, 30-second doctor cockpit, CareShare + break-glass, grounded History AI.

AI-organized from the patient's records -- not medical advice. Decision support only. Synthetic demo data.
Every fact cites its source.

Role	Email	Password
Patient	patient@demo.medcare	demo1234
Doctor	doctor@demo.medcare	demo1234

Emergency OTP (simulated): **123456** | Seed: 15 docs, 6 types, 1 duplicate skipped, Metformin 500-vs-1000 conflict, RELATIVE follow-up, labs x2 providers, 2 hospitals, 1 gap.

Project Guide -- generated from the live codebase

Contents

1. Session Changelog 2
2. Tech Stack 3
3. Architecture 3
4. Data Model (16 Models) 4
5. End-to-End Flow 4
6. API Reference 5
7. Setup & Run 5
8. Testing & Gates 6
9. Project Structure 6
10. Limitations 6

1. Session Changelog

The repo went from 'runs, barely documented' to documented, reproducible and verified:

Commit	Change	Files
c26cd60	Baseline: fixed .pg/e2e.ps1 line-102 bug; app up for E2E (Postgres :5432, seeded DB, Next.js :3001); full guide; .gitignore + remote.	.pg/e2e.ps1, README
3a251f2	Landing: About-this-photo + 3-picture gallery.	app/page.tsx
41f77fb	Landing: How-it-works, Safe-by-design + FAQ, bottom CTA.	app/page.tsx
6c0a4a8	Landing: mobile responsiveness (fluid type, stacked CTAs, 44px targets).	page/layout/css
14fab4a	Docs: professional README.	README.md
38ccb2c	Docs: removed license section.	README.md
3886805	Docs: 3D README styling.	README.md

2. Tech Stack

Layer	Technology	Details
Framework	Next.js 14.2 App Router	SSR pages + Route Handlers as API; middleware guards
UI	React 18, Tailwind 3.4	clinical theme, glass cards; lucide-react icons
Data	PostgreSQL 17 + Prisma 5.22	16 models; DATABASE_URL / DIRECT_URL
Validation	Zod	Upload, extraction, share, emergency, pagination
AI	Pluggable + Mock	providers.ts interfaces; mockProvider.ts rules; AI_PROVIDER=mock
Client state	zustand, TanStack Query	Stores + server-state queries
Viz	reactflow 11, recharts 2, react-pdf 9	Graph, trends, doc viewer
Storage	Local FS storage/documents/	Immutable originals; authorized streams
Auth	Cookie medcare_session	HttpOnly + SameSite=Lax; rbac checks (IDOR-safe)
Tests	Vitest, Node + PS E2E	tests/ + scripts/e2e.mjs + .pg scripts
Tooling	pnpm 9, TS 5.6 strict	@/* alias; ESLint; tsx

3. Architecture

Every page and API route funnels through one guard chain: session cookie - role - object scope. Pages are guarded by **middleware.ts**; API routes re-check server-side so a forged client cannot escalate (no IDOR).

```
Browser (App Router pages)
+- / | /signin | /signup | /forgot
+- /patient/*  overview | journey | documents | ask | share | access-history
+- /doctor/*   dashboard | patients | shared | emergency | audit
+- /share/[token] (public, token-gated view)
    | cookie: medcare_session
    v
middleware.ts -- /patient|/doctor without session -> /signin
    v
API Route Handlers (app/api/*) -- getSession -> requireRole -> rbac scope
+- auth        signup | signin | signout
+- upload      -> storage (sha256 dedupe) -> process pipeline
+- process     -> classify -> extract -> persist
+- timeline | events | documents | snapshots | audit      (scoped reads)
+- documents/[id]/file      (authorized binary stream)
+- chat        -> HistoryAI (grounded answers, refuses advice)
+- share       -> token create | list | revoke
+- emergency   -> break-glass (reason + OTP + 1-hour box)
    v
Services/ -- Timeline | Episode | Extraction | DocumentProcessing
           Conflict | Snapshot | Share | Emergency | HistoryAI
    v
Prisma Client (lib/db.ts) -> PostgreSQL -> storage/documents/* (originals)
```

Layer responsibilities

Layer	Owns	Key files
Presentation	Landing, patient & doctor portals, public share view, glass-card theme	app/page.tsx, app/patient/*, app/doctor/*, app/share/[token], app/globals.css
Edge guard	Redirects unauthenticated page hits	middleware.ts
API surface	Validate input (Zod), resolve session + scope, shape responses	app/api/*/route.ts, lib/api-helpers.ts
Domain services	Timeline, episodes, conflicts/gaps, snapshots, sharing, emergency, AI	services/*.ts
Persistence	Single Prisma client + schema of 16 models	lib/db.ts, db/schema.prisma
File storage	Immutable originals, streamed only after authorization	lib/storage.ts, storage/documents/

Module guide

- **lib/enums.ts** - EvidenceStatus, DatePrecision, EventType (x9), ShareScope (x7), share permission, doc status.
- **lib/validation.ts** - upload metadata (max 15 MB), extraction schema that *requires* evidence_quote + source_page, share create, emergency request.
- **DocumentProcessingService** - CLASSIFYING -> EXTRACTING -> PROCESSED | FAILED, up to 3 attempts, bad AI output quarantined.
- **TimelineService** - precision-first ordering; null-date / fuzzy-band events sorted last.
- **EpisodeService** - 30-day temporal clustering into clinical episodes.

- **ConflictService** - dosage discrepancies + undocumented gaps (over 90 days); conflicts stay OPEN, never auto-resolved.
- **SnapshotService** - What-Changed diffs of conditions, medications, labs, procedures, hospitalizations.
- **HistoryAIService** - retrieval over scoped events, citation building, explicit refusal of medical advice.
- **ShareService** - randomBytes(24) token, HMAC hash stored, hash stripped from every response.
- **EmergencyService** - reason + OTP + confirm gate, 1-hour audited access session.
- **lib/rbac.ts** - access only via PatientDoctor link or a live, unexpired, unrevoked share.

4. Data Model (16 Prisma models)

Identity & access

Model	Purpose	Notable fields
User	Account + role	email (unique), passwordHash, role, auditLogs
PatientProfile	Patient details (1:1)	userId (unique), fullName, dob, gender
DoctorProfile	Doctor details (1:1)	userId (unique), fullName, specialty
PatientDoctor	Care relationship that grants doctor scope	patientId + doctorId (unique pair)

Clinical core

Model	Purpose	Notable fields
Document	Immutable original record	storageKey, sha256, docType, status, failureReason
DocumentPage	Per-page text excerpt	pageNumber, textExcerpt
ClinicalEvent	One traceable clinical fact	eventType, title, eventDate?, datePrecision, sourceDocumentId + sourcePage + evidenceQuote, evidenceStatus, confidence, extractedEntities
EvidenceItem	Extra evidence rows per event	documentId, page, quote
EventRelationship	Typed edge between events	fromEventId, toEventId, type, derivedRule
ClinicalEpisode	30-day cluster of related events	title, startDate, endDate, eventIds
PatientStateSnapshot	State at each ingest (What Changed)	label, data (JSON)
Conflict	Dosage / documentation discrepancy	kind, title, eventIds, status = OPEN
TimelineGap	Undocumented stretch of care	afterEventId, beforeEventId, description

Sharing, emergency & audit

Model	Purpose	Notable fields
ShareSession	Patient-controlled scoped share	tokenHash (unique), scopes[], permission, documentIds[], expiresAt, revoked
EmergencyAccessSession	Audited break-glass window	reason, otpVerified, expiresAt, resourcesAccessed
AuditLog	Append-only activity trail	actorId, actorRole, action, targetType, targetId, accessType, result, meta

Data rules: originals are immutable and addressed by sha256; storageKey never leaves the server. evidenceQuote + sourcePage are mandatory on every event, so a fact without a source cannot be persisted (invariant 'no fabrication').

Total: 16 models - 4 identity/access, 9 clinical core, 3 sharing & audit.

5. End-to-End Flow

The whole product is one pipeline. Each numbered step maps to a route handler and a service.

- **1. Sign in** - POST /api/auth/signin verifies the password and sets the HttpOnly medcare_session cookie.
- **2. Upload** - POST /api/upload accepts PDF / JPG / PNG (max 15 MB), computes SHA-256, skips duplicates, and writes an immutable original to storage/documents/.
- **3. Process** - POST /api/process classifies the document, then extracts events. Dates are mapped faithfully: EXACT only when a full date is present, MONTH for a month-year, RELATIVE stays relative. Output is Zod-validated; failing output becomes FAILED with a reason.
- **4. Persist** - Valid events + evidence rows are written with document id, page and quote. Conflicts, gaps and a snapshot are then recomputed.
- **5. Timeline** - GET /api/timeline returns precision-ordered events; every card can show document + page + quote + evidence status.
- **6. Evidence UI** - Journey cards, evidence badges and panels let the patient open the exact source; document bytes stream only through GET /api/documents/[id]/file after an authorization check.
- **7. What Changed** - Snapshot diffs show what shifted between ingests (new medication, changed dose, new lab, hospitalization, resolved item).
- **8. Doctor cockpit** - GET /api/timeline and /api/audit for a signed-in doctor resolve scope through the PatientDoctor link or a live share; a 30-second view lists conditions, medications, labs, hospitalizations, conflicts and gaps.
- **9. CareShare** - POST /api/share creates an expiring, scoped, revocable link (1h / 24h / 7d / custom). GET lists sessions with the token hash stripped; DELETE revokes; /share/[token] is the public, token-gated view.
- **10. Break-glass** - POST /api/emergency is doctor-only and requires a reason of at least 10 characters, a valid OTP and confirm: true. It opens a 1-hour audited session and records resourcesAccessed.
- **11. History AI** - POST /api/chat retrieves scoped evidence, answers with citations, and refuses dosage or diagnosis advice with a disclaimer plus the records that exist.
- **12. Audit** - GET /api/audit is the append-only trail of logins, uploads, views, shares, revokes and emergencies. Logging never blocks a request.

Invariants I1-I10. (1) every fact is traceable to document + page + quote; (2) enums/badges never invented; (3) no fabrication - invalid AI output is quarantined as FAILED, bad dates become null; (4) date precision is faithful - MONTH is never silently upgraded to a day; (5) originals immutable; (6) no causal claims - adjacency means ordering only; (7) access requires scope or an audited break-glass; (8) shares expire, are revocable and hash-safe; (9) conflicts are surfaced, never auto-resolved; (10) synthetic demo data only.

6. API Reference

Method	Endpoint	Auth	Purpose
POST	/api/auth/signup signin signout	public / any	Register, log in (sets cookie), log out
POST	/api/upload	patient	Store original + create Document row (sha256 dedupe)
POST	/api/process	patient	Classify -> extract -> persist events + evidence
GET	/api/timeline events documents snapshots audit	scoped	Scoped reads; storageKey removed from payloads
GET	/api/documents/[id]/file	scoped	Authorized binary stream of the original
POST	/api/chat	patient	Grounded Q&A with citations; refuses advice
GET / POST / DELETE	/api/share (?id= for DELETE)	patient (own)	List (hashes stripped) / create token / revoke
GET	/share/[token]	token	Public shared view, scope-limited
POST	/api/emergency	doctor	Break-glass: reason + OTP + confirm -> 1-hour session

Response contract

Situation	Status
No session cookie	401 Unauthorized
Cross-patient read, or doctor without a live scope	403 Forbidden (400 where no scope can resolve)
Emergency reason shorter than 10 characters	400 Bad Request
Emergency with a wrong OTP	403 Forbidden
Duplicate upload (same sha256)	handled gracefully - stored once, reported as skipped

Pages **/patient/*** and **/doctor/*** redirect to **/signin** when the cookie is absent; the public landing page, **/signin** and **/signup** stay open. Verified live: unauthenticated API calls return 401, doctor IDOR attempt returns 403, invalid emergency returns 400.

7. Setup & Run

Prerequisites

- Node.js 20 and pnpm 9 (packageManager: pnpm@9.15.1).
- PostgreSQL 17 listening on :5432.
- No AI or OCR API keys are needed - both providers are deterministic mocks.

Commands (Windows / PowerShell)

```
pnpm install
Copy-Item .env.example .env      # then set DATABASE_URL if it differs
pnpm db:generate
pnpm db:migrate
pnpm db:seed
pnpm exec next dev --port 3001
# open http://localhost:3001
```

Gotcha: start the server with **pnpm exec next dev --port 3001**. **pnpm dev -- -p 3001** fails because Next.js treats the extra argument as a project path.

Environment variables

Variable	Example	Purpose
DATABASE_URL / DIRECT_URL	postgresql://medcare:medcare@localhost:5432/medcare?schema=public	Prisma connection
STORAGE_DRIVER / STORAGE_BUCKET	local / medcare-docs	Original-file storage
AI_PROVIDER / OCR_PROVIDER	mock / mock	Deterministic demo (no keys)
SHARE_TOKEN_SECRET	32+ character secret	HMAC for share tokens
EMERGENCY_OTP_MODE / EMERGENCY_OTP	simulated / 123456	Break-glass OTP
APP_URL	http://localhost:3000	Base URL for share links

Seeded demo state: 15 documents across 6 types, 1 duplicate skipped, a Metformin 500-vs-1000 conflict, a RELATIVE follow-up, labs from 2 providers, 2 hospitals and 1 timeline gap.

8. Testing & Quality Gates

```
pnpm typecheck      # tsc --noEmit
pnpm lint           # eslint
pnpm test           # vitest unit suites
pnpm build          # production build
node scripts/e2e.mjs http://localhost:3001
powershell -NoProfile -ExecutionPolicy Bypass -File .pg/e2e.ps1
```

Suite	What it proves
conflict	Dosage-conflict detection and the never-auto-resolve rule
extraction	Schema validity, date-precision fidelity, quote-required
security	401 / 403 guards, hash stripping, OTP + reason gates
temporal	MONTH anchoring, 30-day episodes, 90-day gaps
E2E (33 checks)	Guards, enum validity, traceability, share lifecycle, IDOR, chat grounding

Last verified on this machine: Node E2E 33/33 passed - tsc --noEmit exit 0 - PowerShell E2E (.pg/e2e.ps1) printed DONE - landing page returned 200 on http://localhost:3001.

9. Project Structure

app/	landing + patient/doctor pages + share/[token] + api/* handlers
components/	EvidenceBadge, EventCard, JourneyGraph, LabTrend, DocViewer, ...
lib/	auth rbac enums validation storage audit share-tokens
services/	Timeline Episode Extraction DocumentProcessing Conflict Snapshot Share Emergency HistoryAI
ai/	providers.ts (interfaces) + mockProvider.ts (deterministic demo)
db/	schema.prisma (16 models) + seed.ts (synthetic fixtures)
tests/	conflict extraction security temporal (vitest)
scripts/	e2e.mjs + e2e-smoke.mjs
middleware.ts	/patient /doctor -> /signin without a session
.pg/	local Postgres artefacts (git-ignored) + e2e/smoke scripts
docs/	this guide + its generator scripts

10. Limitations & Roadmap

- **Auth is demo-grade.** Passwords use SHA-256; move to bcrypt/argon2 plus a real session store.
- **AI and OCR are deterministic mocks.** Wire real providers behind ai/providers.ts while keeping the Zod gate and the FAILED quarantine.
- **Storage is local disk.** For multiple instances, switch to object storage with signed URLs.
- **Hardening pending.** Add rate limiting, security headers and refresh-token rotation.
- **Synthetic fixtures only.** Never commit real patient data (PHI).

Contributing

```
git checkout -b feat/my-change
pnpm typecheck; pnpm lint; pnpm test
git commit -m "feat: describe change + E2E result"
```

Keep invariants I1-I10 green and include E2E output in pull requests.

MEDSCOPE-AI - evidence-first by design. Every fact cites its source.

AI-organized from the patient's records - not medical advice. Synthetic demo data.